Getting to Know Spring Boot and REST APIs

Student Workbook 8a – Getting to Know Spring Boot and REST APIs

Version 7.1 Y

Table of Contents

Module 1

A Spring Boot App

Section 1–1

Spring Framework and Spring Boot

Spring Framework

- **Creating enterprise Java applications used to be very complicated before we had frameworks**
  - They had a lot of configuration files
  - Managing object creation was manual and error-prone
  - Connecting components required a lot of repetitive code
- **So, in the early 2000s, a group of people led by Rod Johnson decided to build a framework that would this easier**
  - Makes programming enterprise Java applications much easier
  - Originally for web applications, but works for any Java application
  - Became the most popular Java framework
- **Spring is a framework that is used to manage dependencies between objects in your application**
  - We tell Spring what objects we need and Spring creates them and connects them together
  - We focus on business logic, not object management
- **The core of the Spring Framework is based on two principles:**
  - **Dependency injection** (DI): Spring gives your objects the dependencies they need
    - Instead of MyService service = new MyService(), Spring does it for you
  - **Inversion of control** (IoC)

Spring Boot

- **The spring framework is great, but there is a lot of setup required**
  - Config files to write
  - Dependencies to manage manually
  - Setting up servers was complex
- **Java developers complained, saying they felt like they were XML developers**
- **And that's where Spring Boot (2014) comes in**
  - Takes Spring Framework and adds "convention over configuration"
  - Most things work "out of the box" with sensible defaults
  - You can start coding immediately, not configuring
- **With Spring Boot we use the Spring Framework in a pre-configured way. It comes with:**
  - **Auto-configuration**: Automatically sets up common features
  - **Embedded server**: Tomcat server built right in (no separate installation)
  - **Starter dependencies**: Pre-packaged groups of dependencies
    - Instead of adding 10 libraries, add 1 "starter"
  - **Production-ready features**: Health checks, metrics, etc.

The Demo App

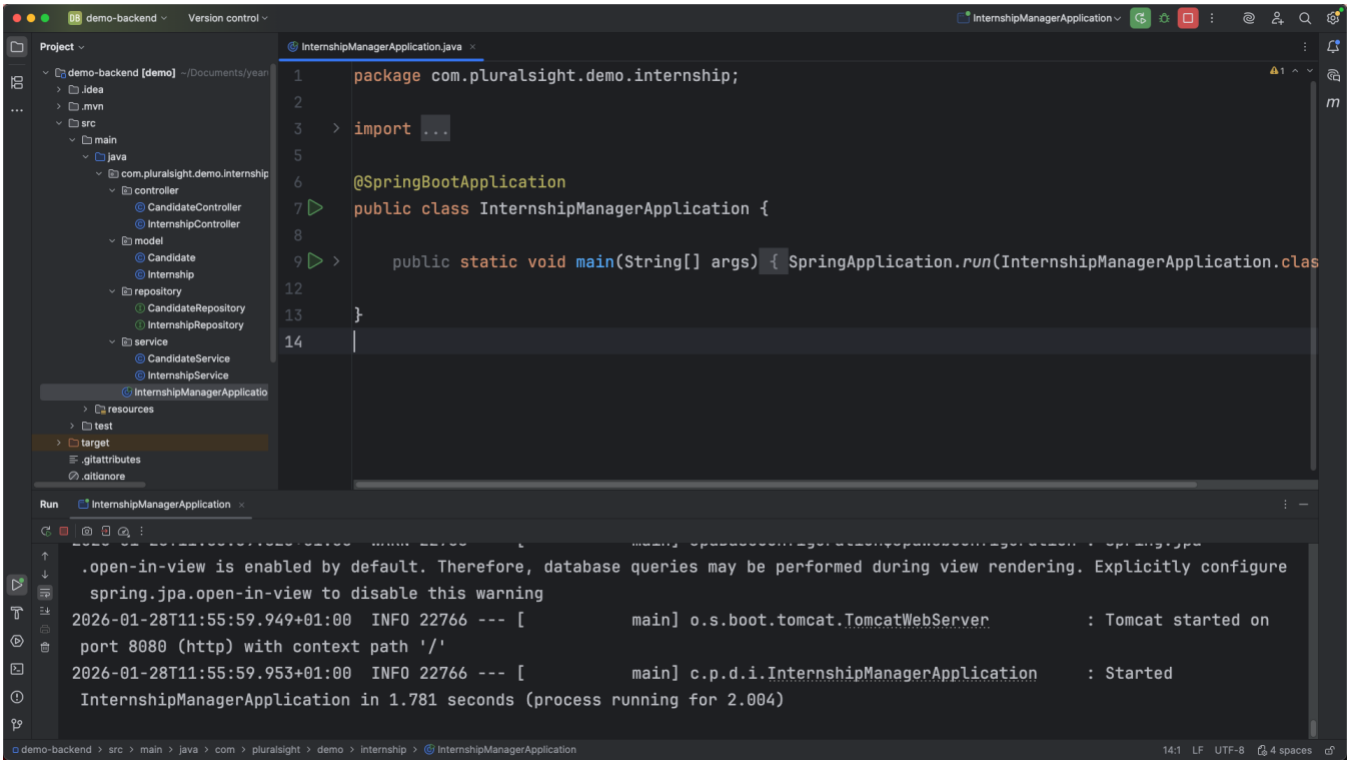
- **To explore Spring Boot, we're going to use a demo Spring Boot application with an internal REST API and a frontend that acts as a platform that connects to this API**

- **It's an API that will**
  - List internship opportunities
  - List candidates
  - Have the ability to add to the list, delete from the list and modifying items on the list

- **This API can be used by applications to create, read, update and delete data**

Backend Demo

- To run the backend API, we have a few different options
- For now, let's open the project in IntelliJ, locate the class with the main method and hit the run button



- This might not work on your computer yet, due to configuration issues (don’t worry, we’ll get there)
- The working example is connecting to a MySQL database (that’s why it might not work on your machine yet)

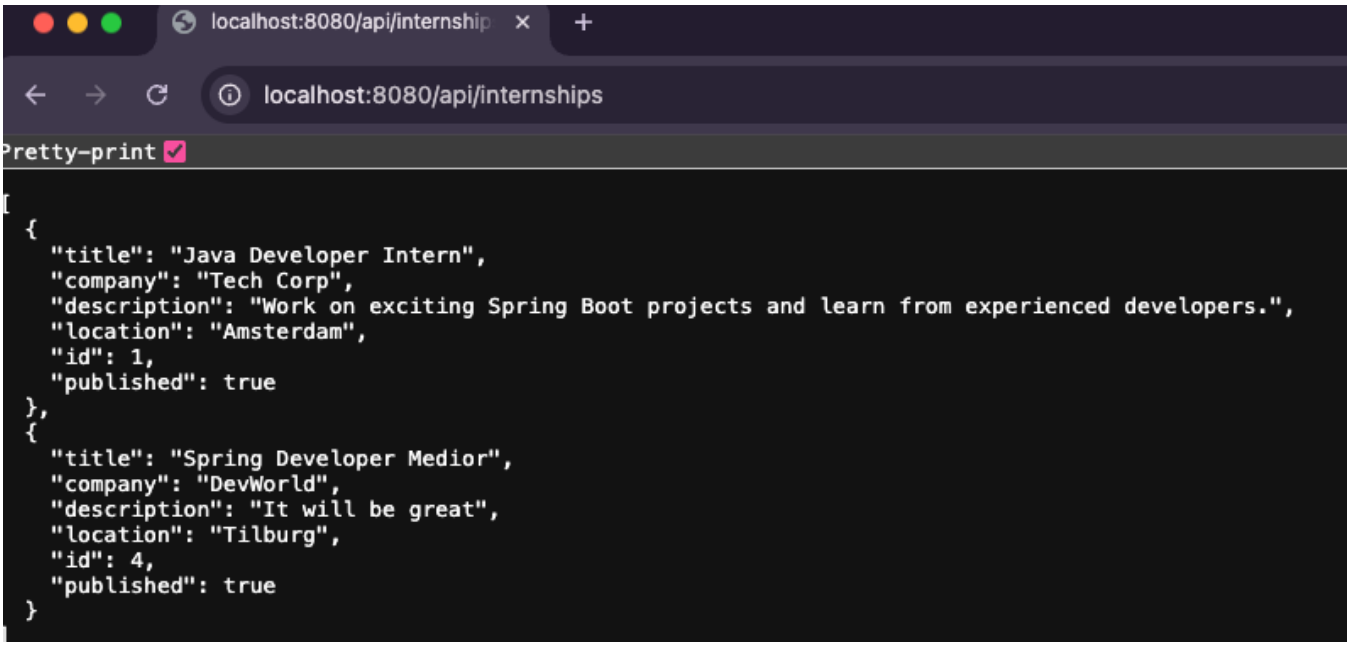
Testing the Backend

- **We have multiple options to see if the backend is working**
  - Applications like Postman and Insomnia
    - (in this class we will use Insomnia)
  - Testing the get endpoints in the browser (**GET** endpoints are for reading data)
  - Using curl on the command line
  - Requesting from another application (uncommon for testing, but it will give us the information we need)

- **Let's start by going to this url: http://localhost:8080/api/internships**

- **If your database is empty, you should see empty brackets [ ] on the web page**

  – An example when you have data in your database:



Exercise – Spinning up the Backend

In this exercise, you'll get the demo Spring Boot application running on your machine. You'll configure the database connection, start the application, and verify it works by calling an API endpoint in your browser.

STEP 1 - Open the project

- Unzip the provided project file
- Open the unzipped folder in IntelliJ as a project
- Take a moment to look at the project structure. You should see packages for controller, model, repository, and service under src/main/java

STEP 2 – Seed the database

- Open MySQL Workbench (or your preferred MySQL client)
- Open the file `setup.sql` from the project
- Run the SQL script. This will create the internships database and the tables the app needs

STEP 3 - Configure the database connection

In IntelliJ, open the file `src/main/resources/application.properties`

Find these two lines:

```
spring.datasource.username=root
spring.datasource.password=root
```

Change `root` to your own MySQL username and password

Don't change anything else in this file for now

STEP 4 - Run the application

In the project, find the file `InternshipManagerApplication` (it's in the root of the package, not inside a sub-package)

This file contains the main method. Click the green play button next to it, or right-click and select Run

Watch the console output! You should see Spring Boot starting up. Look for a line that says something like: `Started InternshipManagerApplication`

If you see errors about the database connection, double-check your username and password in `application.properties`

STEP 5 - Test that it works

Open your browser and go to: `http://localhost:8080/api/candidates`

You should see `[]`. This is an empty JSON list. This means the app is running and connected to the database, there's just no data yet!

STEP 6 - Add some test data

- Go back to `setup.sql`
- Find the `INSERT INTO` statements that are commented out
- Uncomment them and run them in your MySQL client
- Now refresh your browser at `http://localhost:8080/api/candidates`
- You should now see the candidate data coming back as JSON

If you see JSON data in your browser, your backend is up and running!

Create a new project on Github for this project, commit and push your code!

JSON

- **Our backend API responds with data in JSON format, this is usually the case for REST APIs**

- **JSON stands for JavaScript Object Notation**

- **It's a way to translate objects into text, so they can be sent over the network**

- **Here's an example of a simple JSON object holding a very basic candidate object**

Example

```
{
  "name": "Zia"
}
```

- **Let's dissect that example:**
  - `{ }` means: this is an object
  - `"name"` is a key
  - `"Zia"` is a value of type text (string)
  - Keys are always strings
  - Values can be text, numbers, true/false, lists, or other objects

Example

```
{
  "name": "Zia",
  "age": 29,
  "available": true
}
```

- **The one above is slightly more complicated, because it holds multiple key-value pairs. Some other things to note:**
  - Numbers don't use quotes (`29`)
  - Booleans are `true` or `false` (not `"true"`)
  - Each key-value pair is separated by a comma

Example

```
{
  "id": 1,
  "name": "Zia Jansen",
  "address": {
    "street": "Main Street 12",
    "city": "Utrecht",
    "country": "Netherlands"
  }
}
```

- **This one above is holding a nested object**
  - The `address` is not text, but is in another JSON object
  - Objects can contain other objects

- **JSON can also hold lists of data, this is indicated by [ ]**

- **All values inside the list are separated by commas**

```
{
  "id": 1,
  "name": "Zia Jansen",
  "skills": ["Java", "Spring Boot", "SQL"]
}
```

- **This nesting can be many levels deep**

```
{
  "id": 1,
  "name": "Zia Jansen",
  "skills": [
    {
      "name": "Java",
      "level": "advanced"
    },
    {
      "name": "Spring Boot",
      "level": "intermediate"
    }
  ]
}
```

Exercises – Using JSON

EXERCISE 1 - Reading JSON

Below is a JSON object representing a company. Study it carefully, then answer the questions that follow.

```
{
  "company": "SpringBootAppBuilders",
  "founded": 2019,
  "active": true,
  "headquarters": {
    "street": "124 Pineapple Lane",
    "city": "Bikini Bottom",
    "country": "Pacific Ocean"
  },
  "departments": [
    {
      "name": "HR",
      "floor": 1,
      "employees": [
        {
          "name": "SpongeBob SquarePants",
          "age": 34,
          "jobTitle": "HR Manager",
          "fullTime": true,
          "phone": "+1-555-0101",
          "address": {
            "street": "124 Conch Street",
            "city": "Bikini Bottom",
            "zipCode": "BB-1001"
          },
          "hobbies": ["jellyfishing", "karate", "bubble blowing", "cooking"],
          "interests": ["marine biology", "employee happiness"]
        },
        {
          "name": "Gary the Snail",
          "age": 28,
          "jobTitle": "HR Assistant",
          "fullTime": false,
          "phone": "+1-555-0102",
          "address": {
            "street": "124 Conch Street",
            "city": "Bikini Bottom",
            "zipCode": "BB-1001"
          },
          "hobbies": ["napping", "poetry"],
          "interests": ["snail care", "classical music"]
        }
      ]
    },
```

```
    {
      "name": "IT",
      "floor": 3,
      "employees": [
        {
          "name": "Sandy Cheeks",
          "age": 36,
          "jobTitle": "Lead Developer",
          "fullTime": true,
          "phone": "+1-555-0201",
          "address": {
            "street": "Treedome Drive 1",
            "city": "Bikini Bottom",
            "zipCode": "BB-2001"
          },
          "hobbies": ["karate", "science experiments", "surfing", "rodeo"],
          "interests": ["artificial intelligence", "robotics", "Texas BBQ"]
        },
        {
          "name": "Squidward Tentacles",
          "age": 45,
          "jobTitle": "Senior Developer",
          "fullTime": true,
          "phone": "+1-555-0202",
          "address": {
            "street": "122 Conch Street",
            "city": "Bikini Bottom",
            "zipCode": "BB-1003"
          },
          "hobbies": ["clarinet", "painting", "sculpture"],
          "interests": ["fine arts", "architecture", "peace and quiet"]
        },
        {
          "name": "Karen Plankton",
          "age": 38,
          "jobTitle": "Database Administrator",
          "fullTime": true,
          "phone": "+1-555-0203",
          "address": {
            "street": "1 Chum Bucket Way",
            "city": "Bikini Bottom",
            "zipCode": "BB-3001"
          },
          "hobbies": ["data analysis", "chess"],
          "interests": ["machine learning", "world domination strategies"]
        },
        {
          "name": "Larry the Lobster",
          "age": 32,
          "jobTitle": "Junior Developer",
          "fullTime": true,
          "phone": "+1-555-0204",
          "address": {
            "street": "15 Mussel Beach Road",
            "city": "Bikini Bottom",
```

```
            "zipCode": "BB-4001"
          },
          "hobbies": ["weightlifting", "swimming", "volleyball"],
          "interests": ["fitness apps", "sports technology"]
        }
      ]
    },
    {
      "name": "Sales",
      "floor": 2,
      "employees": [
        {
          "name": "Patrick Star",
          "age": 35,
          "jobTitle": "Sales Representative",
          "fullTime": true,
          "phone": "+1-555-0301",
          "address": {
            "street": "120 Conch Street",
            "city": "Bikini Bottom",
            "zipCode": "BB-1002"
          },
          "hobbies": ["sleeping", "eating ice cream", "watching TV"],
          "interests": ["rocks", "doing nothing professionally"]
        },
        {
          "name": "Mr. Krabs",
          "age": 57,
          "jobTitle": "Sales Director",
          "fullTime": true,
          "phone": "+1-555-0302",
          "address": {
            "street": "3541 Anchor Way",
            "city": "Bikini Bottom",
            "zipCode": "BB-5001"
          },
          "hobbies": ["counting money", "treasure hunting", "sailing"],
          "interests": ["profit margins", "cost reduction", "gold"]
        },
        {
          "name": "Pearl Krabs",
          "age": 23,
          "jobTitle": "Sales Intern",
          "fullTime": false,
          "phone": "+1-555-0303",
          "address": {
            "street": "3541 Anchor Way",
            "city": "Bikini Bottom",
            "zipCode": "BB-5001"
          },
          "hobbies": ["cheerleading", "shopping", "social media"],
          "interests": ["marketing trends", "fashion"]
        }
      ]
    }
```

```
  ]}
```

Write down the answer and the path you followed to find it (e.g. company → departments → first department → name)

- What year was the company founded?
- How many departments does the company have?
- On which floor does the IT department sit?
- How many employees work in Sales?
- What is SpongeBob's third hobby?
- Who lives at 122 Conch Street?
- What is Karen Plankton's job title?
- Which employees are not full-time?
- Who shares the same address? Why might that be?
- What is Patrick Star's second interest?
- How many hobbies does Sandy Cheeks have?
- What is the zip code of the employee who plays clarinet?
- Who is the youngest employee across all departments?
- Which department has the most employees?
- What type is the value of active at the top level? E.g. is it a string, a number, or something else?

EXERCISE 2 - Writing JSON

Now it's your turn to write JSON. We'll build it up step by step.

Step 1 - A simple object

Create a JSON object that represents your best friend. Include the following key-value pairs:

- Their first name (text)
- Their age (number)
- Their city (text)
- Whether they like coffee (true/false)

For example, the structure should look something like:

```
{
  "firstName": "...",
  "age": ...,
  "city": "...",
  "likesCoffee": ...
}
```

Step 2 - Add a list

Add a key called `"favoriteMovies"` to your object. The value should be a list of 3 movies (as text)

Step 3 - Add a nested object

Your friend has a pet. If they don't have one in real life, make one up. Add a key called "pet" to your object. The value should be a nested object with:

- name (text)
- animal (text, e.g. "cat", "dog", "hamster")
- age (number)
- vaccinated (true/false)

Step 4 - Bring it all together

Check your complete JSON object. It should now have simple values, a list, and a nested object. Verify that:

- All strings are in double quotes
- Numbers and booleans are not in quotes
- Every key-value pair is separated by a comma
- There is no comma after the last item in an object or list

To make sure your JSON is correct, you can paste it in here (**never do this with real client data!):** https://jsonformatter.curiousconcept.com/

Step 5 - Create a list of objects

Now create a new JSON structure: a list of 3 friends or colleagues. Each person in the list should have:

- firstName
- age
- city
- favoriteMovies (a list of at least 2)
- pet (a nested object with name, animal, and age)

Remember: a list of objects starts with `[` and ends with `]`, with each object separated by a comma. Again, to make sure your JSON is correct, you can paste it in here (still never do this with real client data!): https://jsonformatter.curiousconcept.com/

Section 1–2

Understanding the Structure

Overview of the Structure

- **Our app, like many REST API's are organized into layers, we call this a layered architecture**

- **Every layer has a specific job**

- **These are the four main layers:**
  - Controller
  - Service
  - Repository
  - Model

- **Here’s how the data of a request flows when you want to add an internship:**
  - Frontend sends HTTP POST request
  - Controller receives it: "Someone wants to create an internship"
  - Service processes it: "Let me check the rules and apply business logic"
  - Repository saves it: "I'll talk to the database"
  - Model defines it: "Here's what an internship looks like"
  - Response flows back up: Repository → Service → Controller → Frontend

Controllers

- **Controllers handle the HTTP requests and responses**
- **They receive requests from clients (frontend, mobile apps, Insomnia, other backends)**
- **Sends back responses (often JSON data)**
- **What the controller does:**
  - Defines API endpoints (URLs)
  - Parses incoming data
  - Calls the service layer
  - Returns proper HTTP status codes

Example: CandidateController

- **Here's an example of a controller in our demo API. You don't have to understand all of it at this point**

- **On top of the class, there are annotations (starting with @).**

- **Annotations are a Java concept, not exclusive to Spring (e.g. `@FunctionalInterface` and `@Override`). They're metadata.**

- **Here we have Spring annotations add Spring magic to the class and make it function like a controller and give it a certain URL**

```
@RestController
@RequestMapping("/api/candidates")
@CrossOrigin(origins = "*")
public class CandidateController {

    private final CandidateService cs;

    public CandidateController(CandidateService cs) {
        this.cs = cs;
    }

    @GetMapping
    public ResponseEntity<List<Candidate>> getAllCandidates() {
        List<Candidate> candidates = cs.getAllCandidates();
        return ResponseEntity.ok(candidates);
    }

    // other endpoints omitted
}
```

- **The methods also have annotations like `@GetMapping`, this is how the methods can be triggered by clients**

Understanding Controllers

- **Controllers don't contain business logic**
- **Instead, in the methods, they will delegate any business logic to a service**
- **You can see that the service is not instantiated, but is passed in via the constructor**
- **Spring creates the controller when the application starts, and automatically provides the service**
  - (Spring also creates the service when the application starts, we will discuss more on that later)
- **This will provide the service to the controller**
- **And this is called <u>dependency injection</u>**
  - the controller depends on the service, hence the service is a dependency

Services

- **A service contains the business rules and logic**
- **It makes decisions based on the business requirements**
  - for example, only published internships should be visible, validates data and handles exceptions
- **It's the layer coordinating between the controller and repository**
- **To tell Spring that it's a service, the class is annotated with `@Service`**
- **The service often depends on a repository**
  - this is a dependency and it's injected by Spring (often via the constructor)
- **The methods in the service are typically used by the controller to trigger the business logic**
- **The methods in the service are in charge of communicating with the repository to fetch and store data**
- **Services are important for code reusability and testability**
- **Controllers call services, services can call other services, and they call themselves**

Example: CandidateService

- **Here's a part of the demo service:**

```
@Service
public class CandidateService {

    private final CandidateRepository cr;

    public CandidateService(CandidateRepository cr) {
        this.cr = cr;
    }

    public List<Candidate> getAllCandidates() {
        return cr.findAll();
    }

    public Candidate getCandidateById(Long id) {
        // Intentional flaw: throws RuntimeException
        return cr.findById(id)
                .orElseThrow(() -> new RuntimeException("Candidate
not found with id: " + id));
    }

    public Candidate createCandidate(Candidate candidate) {
        return cr.save(candidate);
    }

    public Candidate updateCandidate(Long id, Candidate updated) {
        Candidate existing = getCandidateById(id);
        existing.setName(updated.getName());
        existing.setEmail(updated.getEmail());
        existing.setFieldOfStudy(updated.getFieldOfStudy());
        return cr.save(existing);
    }

    public void deleteCandidate(Long id) {
        cr.deleteById(id);
    }
}
```

Models

- **The model defines the structure of your data**
- **It represents the database tables as Java classes and will map the Java fields to database columns**
- **We call those classes “entities” in JPA (Java Persistence API)**
- **Models can contain fields, getters, setters, constructors and relationships between entities**
- **Common annotations for models:**
  - `@Entity`: This class maps to a database table
  - `@Table(name = "...")` can be used to specify table name
  - `@Id` on a field means that this field is the primary key
  - `@GeneratedValue` means that the database generates this value automatically
  - `@Column` is for customizing column behavior (such as renaming)

Example Model: Candidate

- **Here you can see an example of a model Candidate**

```
@Entity
@Table(name = "candidates")
public class Candidate {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private String fieldOfStudy;

    // Constructors
    public Candidate() {
    }

    public Candidate(String name, String email, String fos) {
        this.name = name;
        this.email = email;
        this.fieldOfStudy = fos;
    }

    // Getters and setters omitted

}
```

- **This what a candidate looks like in the database**

id	email	field_of_study	name
1	jan.devries@example....	Computer Science	Jan de Vries

Repositories

- **Repositories handle the database operations**
- **The CRUD operations - create, read, update, delete**
- **They abstract the usage of SQL, which means you won't be writing basic SQL with JPA**
- **Spring Data JPA generates the implementations for you**
  - (including the JDBC / SQL that comes with that)
- **SQL can still be used for more complex queries**
- **Repositories don't involve too much complexity**
- **Just make an interface that extends `JpaRepository<EntityType, IdType>` and done!**
- **`JpaRepository` comes with built-in methods such as:**
  - `findAll()`
  - `findById(id)`
  - `save(entity)`
  - `deleteById(id)`
  - `count()`
  - `existsById(id)`

Example: CandidateRepository

- **Here's an example repository**

```
@Repository
public interface CandidateRepository extends
JpaRepository<Candidate, Long> {
    // No custom queries yet
}
```

- **Here's what's happening:**
  - `@Repository` tells Spring to manage this as a repository component
  - `Interface` means we only define the contract, not the implementation
  - `extends JpaRepository<Internship, Long>`
    - `Internship`: The entity type
    - `Long`: The ID field type

- **Spring Data JPA creates the class at runtime because of the extension! This class is called a runtime proxy implementation**

- **SQL is generated by even more magic**

- **For example, when you see the method `save()`, an `INSERT INTO` query is generated**

- **The repository is used by the service**

Exercise – Draw the Architecture

In this exercise, you'll let your inner Picasso out and draw the layered architecture of the demo application. The goal is to make the invisible structure visible. By drawing it, you'll better understand how the different parts of a Spring Boot application are connected.

What to draw

On a piece of paper (or an (online) whiteboard, or paint), draw the four layers of the application and show how they relate to each other.

Include:

- The name of each layer (Controller, Service, Repository, Model)
- The direction of communication: which layer talks to which?
- For each layer, write down in your own words what its job is (one sentence is enough)
- The actual class names from the demo project that belong in each layer (think about both `Internship` and `Candidate`)
- Where the database fits in the picture
- Where the client (browser, frontend, Insomnia) fits in the picture

Hints to get started

- Think of it as a vertical stack. Requests come in at the top and travel down toward the database. Responses travel back up.
- Ask yourself: if I click a button on the frontend, which class receives that action first? What does that class do with it? Where does the data go next?
- The Model is a bit different from the other three as it doesn't sit in the flow of a request. Instead, it's used by the other layers. Think of it as the shared language they all speak. How would you represent that in your drawing?
- Look at the project structure screenshot from IntelliJ if you need a reminder of which classes exist.

Section 1–3

APIs Explained

APIs

- **API stands for Application Programming Interface**
  - These interfaces aren't exactly designed for a user's experience; They are built to help facilitate the interaction between programs
- **This means that they enable programs to talk to each other**
- **The restaurant analogy:**
  - You (a customer) make an order with the waiter (API)
  - The waiter communicates this to the kitchen (backend)
  - Your order is a *request*
  - The kitchen preparing the food is the *response*
  - As a customer, you don't see how the order is made (hidden implementation)
  - The waiter brings you the food when it's ready
- **APIs are like the waiter. They are the interface the client can communicate with**
- **We use APIs for many reasons**
  - Your phone is likely to use the weather API to display information to you about the weather
  - Your car is likely to use a traffic API to give you information about the traffic

APIs

- **We need APIs for different reasons. One example would be exposing information in a safe way to external services**

- **We often separate the frontend and backend; the frontend then communicates with the backend via the backend's API**

- **Multiple clients can be using the same backend (think about the weather API, your phone can talk to it, boards outside can communicate with it, it could be integrated in a news website, and more)**

- **APIs are a controlled and safe way to expose information, it's possible to only disclose information to authenticated and authorized clients**

- **REST APIs are a specific type of API:**
  - Use HTTP for communication
  - Use URLs to identify resources
  - Use HTTP methods (GET, POST, PUT, DELETE) for specific actions
  - Response is in JSON format

- **REST APIs are today's standard for building APIs**

REST API Endpoints

- **Our demo has several API endpoints, for example:**
  - GET /api/internships → Get all internships
  - GET /api/internships/{id} → Get one internship with a specific id
  - POST /api/internships → Create new internship
  - PUT /api/internships/{id} → Update an existing internship
  - DELETE /api/internships/{id} → Delete an internship with a specific id
- **Can you guess what the URL for getting one candidate by id looks like?**
- **API requests return a response that consists of three parts:**
  - The data body (often JSON)
  - Status code
  - Headers (metadata, such as the type of the content in the body)
- **Examples of HTTP Status Codes**
  - 200 OK: Success! Here's your data
  - 201 Created: Success! New resource created
  - 204 No Content: Success! No data to return
  - 400 Bad Request: You sent invalid data
  - 404 Not Found: Resource doesn't exist
  - 500 Internal Server Error: Something broke on the server

Controller Endpoints to URLs

- **The controller creates the URLs that can be called, these are referred to as the API endpoints**

- **In the below example, there are 2 endpoints:**
  - GET /api/candidates → Get all candidates
  - GET /api/candidates/{id} → Get one candidate using an id

```
@RestController
@RequestMapping("/api/candidates")
@CrossOrigin(origins = "*")
public class CandidateController {

    private final CandidateService cs;

    public CandidateController(CandidateService cs) {
        this.cs = cs;
    }

    @GetMapping // no extra word, so no extra URL parts
    public ResponseEntity<List<Candidate>> getAllCandidates() {
        List<Candidate> candidates = cs.getAllCandidates();
        return ResponseEntity.ok(candidates);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Candidate> getCandidateById(
     @PathVariable Long id
    ) {
        Candidate candidate = cs.getCandidateById(id);
        return ResponseEntity.ok(candidate);
    }

    // other endpoints omitted
}
```

Spring Magic HTTP Requests

- **When creating a controller like this, a lot is happening behind the scenes**
- **You don't need to understand all of what is happening to use it**
- **To give you an idea of what is happening:**
  - Client sends HTTP request
  - Spring receives it and looks at URL + method
  - Spring finds matching controller method
  - Spring converts JSON to Java objects (if needed)
  - Your method executes
  - Spring converts Java objects back to JSON
  - Spring sends HTTP response with proper status code

Exercises – Calling APIs

In this exercise, you'll explore APIs by calling real ones on the internet, discovering hidden API calls on websites you use every day, and connecting it all back to your own running backend.

EXERCISE 1- Calling public APIs in your browser

There are thousands of free, public APIs on the internet that anyone can call. Since GET requests are just asking for data, you can make them directly in your browser. All you need to do, is just type the URL in the address bar and hit Enter.

Try the following:

1. Open your browser and go to: https://catfact.ninja/fact
    - You should see a JSON response with a random cat fact
    - Refresh the page a few times. Do you get different results?
2. Now try these:
    - https://official-joke-api.appspot.com/random_joke and get a random joke
    - https://api.agify.io?name=maaike . This one predicts age based on a name (try your own name!)
    - https://dog.ceo/api/breeds/image/random to get a random dog image URL
3. For each API, answer these questions:
    - What keys does the JSON response contain?
    - What types are the values (string, number, boolean, list, object)?
    - Can you spot the difference between an API that returns a single object and one that returns a list?
4. Find your own: Search online for "free public APIs" and find one that interests you. Call it in your browser and examine the JSON it returns.

EXERCISE 2 - Discovering API calls on real websites

Every time you visit a website, the frontend is often making API calls to a backend behind the scenes and you just don't see them. Let's change that using your browser's Developer Tools.

Opening Developer Tools

In order to do that, we need to open up the DevTools first.

	Shortcut
Windows / Linux	Press F12 or Ctrl + Shift + I
macOS	Press Cmd + Option + I

Alternatively, right-click anywhere on a page and choose **Inspect**.

Now that the Developer Tools are open, and click the **Network** tab

In the filter bar, click **Fetch/XHR**. This filters out images, stylesheets, and scripts, showing only API calls.

Now navigate to a website you use regularly (for example a news site, a webshop, or a streaming service) or reload the page if you're already there.

You should see requests appearing in the list. These are the API calls the frontend is making!

Click on any request and explore:

- Look at the **Headers** tab — can you find the request URL and the HTTP method (it should say GET)?
- Look at the **Response** tab — can you see JSON data? This is exactly what the frontend received from the backend
- Look at the **Preview** tab — this shows the same JSON in a more readable, collapsible format

Try this on 2 or 3 different websites. You'll be surprised how many API calls are happening without you knowing!

Think about what you're seeing:

- What kind of data is being loaded? Product info? User data? Recommendations?
- Can you match what you see in the JSON response to something visible on the page?

EXERCISE 3 – Calling your own API

Now let's bring it back to your own application.

1. Make sure your Spring Boot backend is running (if not, start it again from IntelliJ)
2. Open your browser and go to: http://localhost:8080/api/candidates
3. Now open Developer Tools on that same page, go to the **Network** tab, filter by **Fetch/XHR**, and refresh
4. Click the request that appears and look at the Response tab

Notice how this is the exact same pattern you just saw on real websites: a client (your browser) makes a GET request to a URL, and a backend returns JSON. The only difference is that this backend is running on your own machine instead of somewhere on the internet.

5. Also try: http://localhost:8080/api/internships
6. Compare the structure of the JSON your API returns with the public APIs you called in Exercise 1. What similarities do you see?

Section 1–4

The Frontend and Other Clients

Frontend

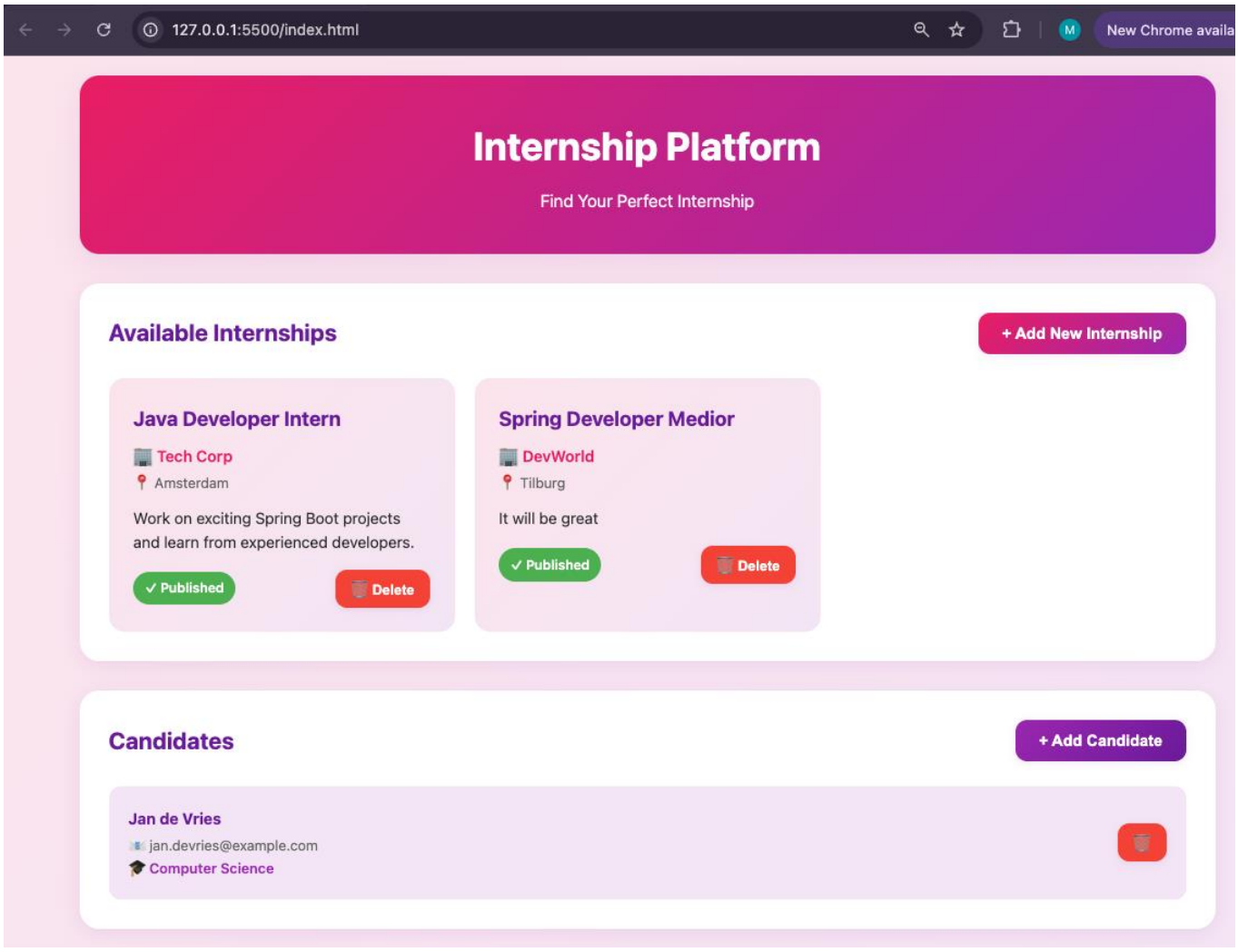
- **The frontend is the visual interface users interact with (so for example a website you see in your browser)**

- **Let's say you want to log in to a web portal**
  - You see the form to log into on the user interface
  - After filling it out and clicking send, the frontend sends a request to the backend (using the API)
  - The backend then processes the response and sends this back to the frontend

- **Many things you see on the frontend are coming from API calls to the backend, for example:**
  - Products in a webshop
  - Courses in the Pluralsight library
  - Details on one specific course
  - Categories of courses in Pluralsight or categories of products on the webshop
  - News articles on a news website
  - And so much more!

Demo Frontend

- **For the demo backend, there is also a demo frontend**
- **This frontend uses the backend endpoints (API) to**
  - Get all internships and candidates
  - Allow adding, deleting and modifying internships and candidates

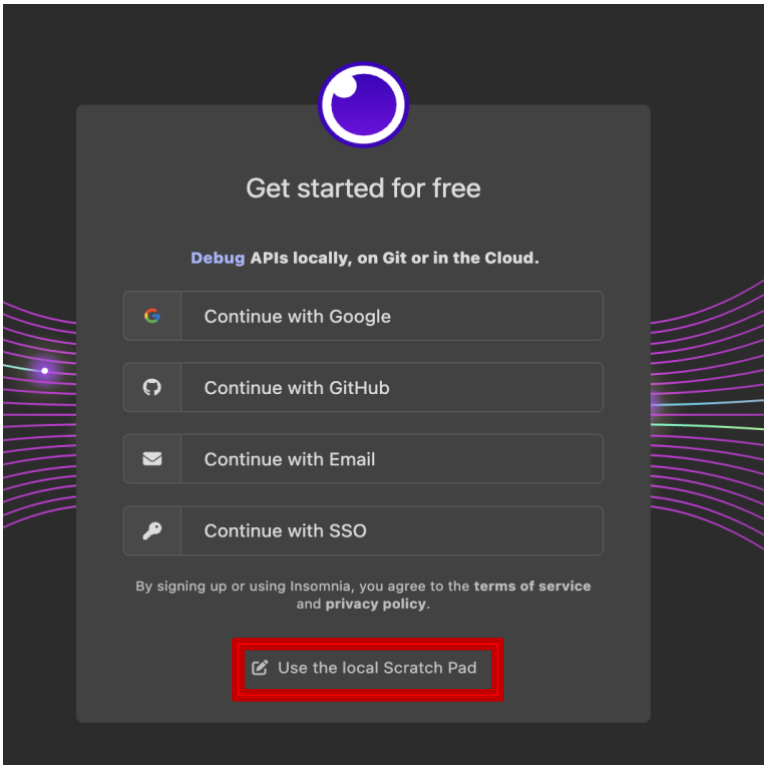


What is an API Testing Tool?

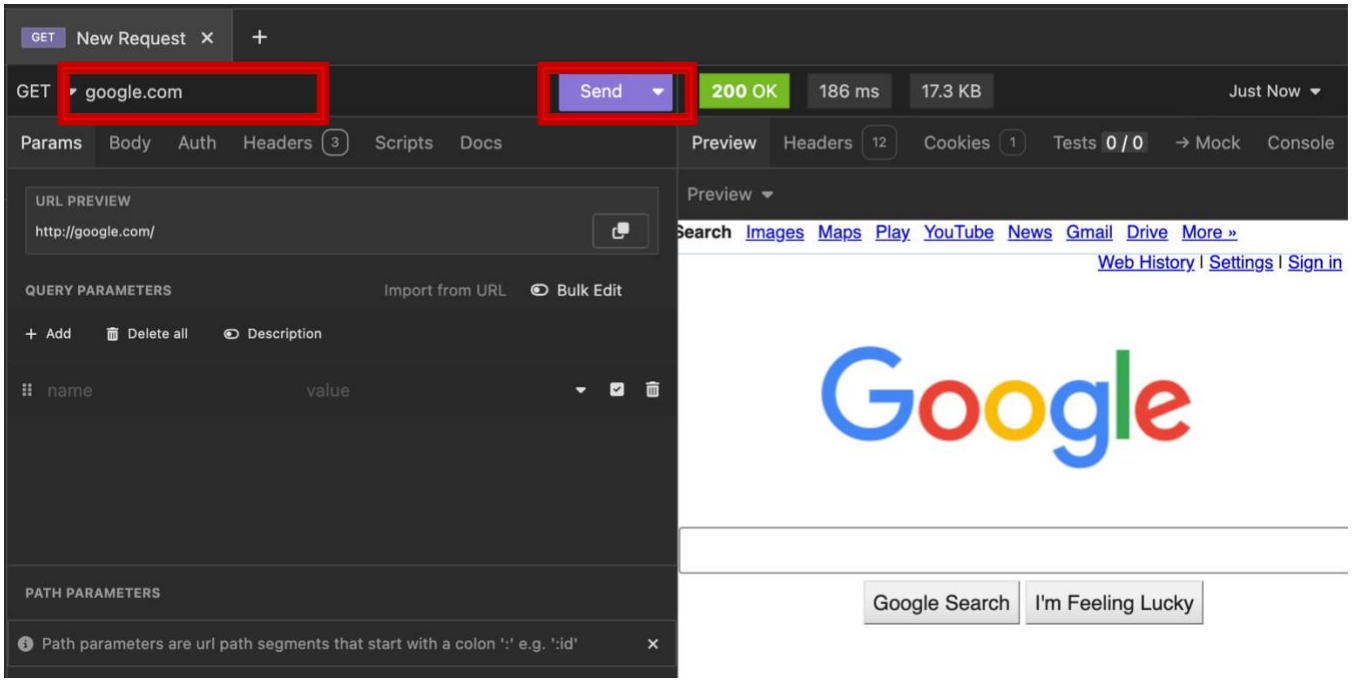
- **RESTful services support GET, POST, PUT and DELETE verbs**
- **Browsers only directly support GET and POST methods**
  - This makes it difficult to test your RESTful API with a browser
    - You could create a web application with JavaScript to test the API
    - You could create a desktop application using Java, C# or another programming language
- **There are many tools that provide you with the tools necessary to test API endpoints**
  - Such as Postman, Insomnia, or Swagger.io
- **We will use Insomnia in this course**
  - Use a search engine to learn about the other API testing tools

Using Insomnia

- **To test an API, when you first open the app, you will need to click ‘Use the local Scratch Pad’**



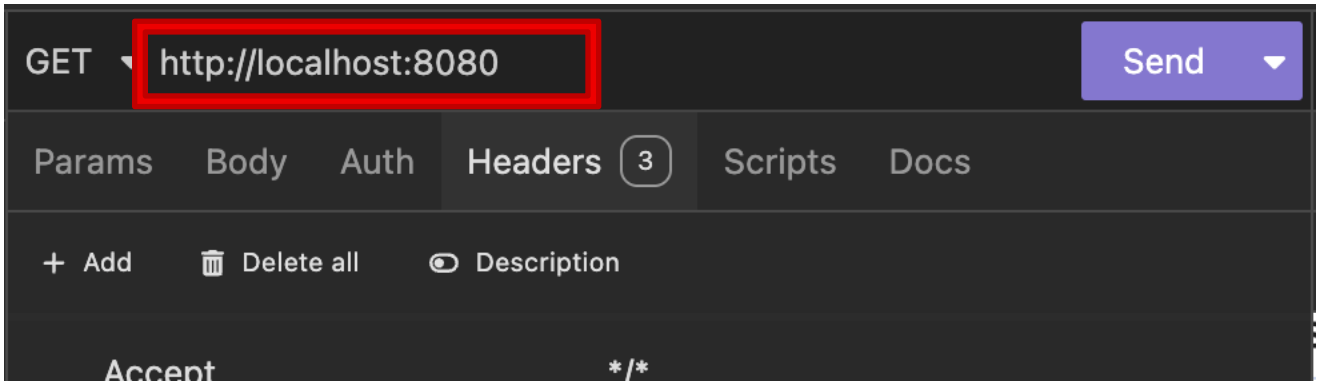
- **Next, enter your API url and hit ‘Send’**



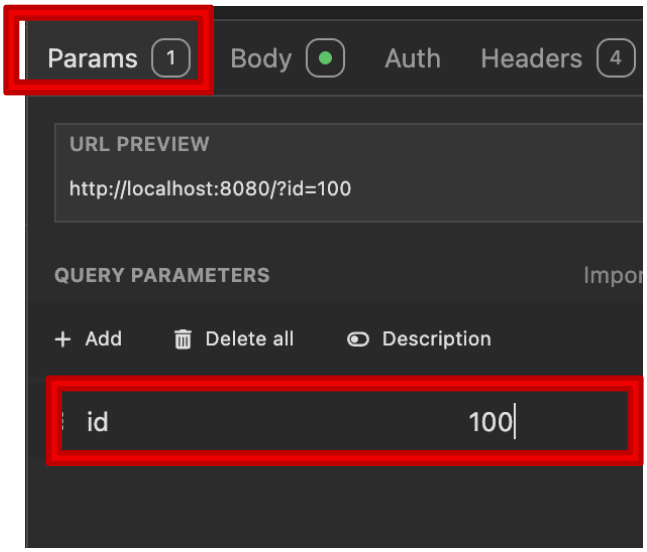
Making the Request

- **An HTTP request in Insomnia is similar to navigating to a website in your browser**

- **For a basic request just add the address (URL) of your API**
  - Ensure that the API is running before you click Send

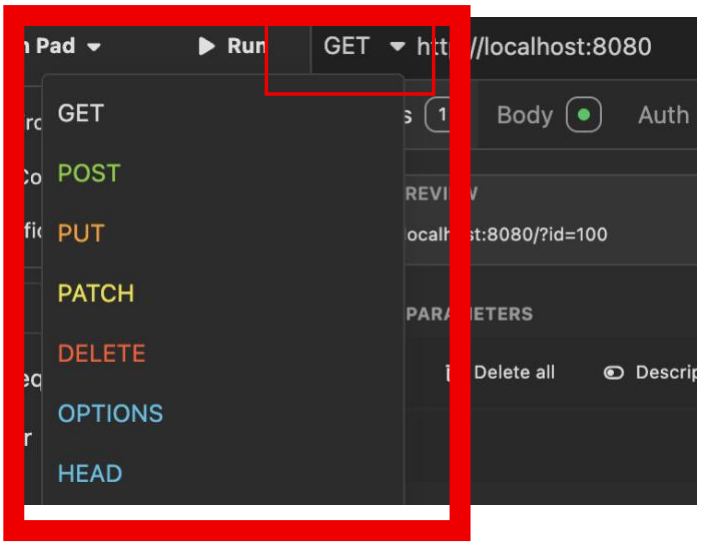


- **Below the URL you can configure the information that you want to SEND TO the server with your request**



Specifying the Verb

- **Insomnia allows you to specify the verb of the request**
  - GET, POST, PUT, DELETE etc.

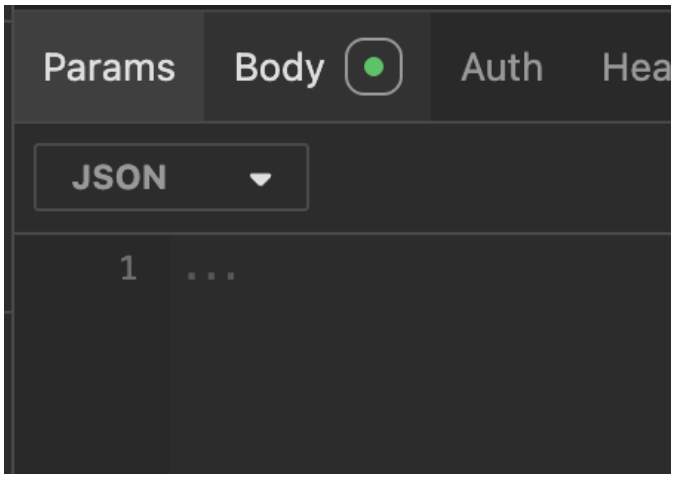


- **As we develop APIs throughout this workbook, you will use various verbs with Insomnia**

Adding a Body to the Request

- **`POST` and `PUT` requests are accompanied by a request body**

- **To add a body select the Body tab and enter the data**
  - To send JSON you must set the body type to raw
  - Select JSON from the text type drop down
  - Enter the JSON data in the text



Exercise – Calling the Backend with Insomnia

In this exercise, you'll use Insomnia to make all types of HTTP requests to your running backend. For each request, pay attention to how the URL and HTTP method map to a specific method in the controller.

Make sure your Spring Boot backend is running before you start.

EXERCISE 1 - GET all candidates

This calls the `getAllCandidates()` method in the controller, which is annotated with `@GetMapping` (no extra path). Combined with the class-level `@RequestMapping("/api/candidates")`, this gives us the full URL.

Setting	Value
Method	GET
URL	http://localhost:8080/api/candidates
Body	none

- Click **Send**
- You should see a JSON list of all candidates
- If the database is empty, you'll see `[]`

EXERCISE 2 - GET one candidate by ID

This calls the `getCandidateById()` method, annotated with `@GetMapping("/{id}")`. The `{id}` part is a path variable — you replace it with an actual number. The `@PathVariable Long id` in the method signature tells Spring to extract that number from the URL.

Setting	Value
Method	GET
URL	http://localhost:8080/api/candidates/1
Body	none

- Click **Send**
- You should see a single candidate object
- Try changing the 1 to another number — what happens if you use an ID that doesn't exist?

EXERCISE 3 - POST a new candidate

This calls the `createCandidate()` method, annotated with `@PostMapping`. Notice `@RequestBody Candidate candidate` in the method signature — this tells Spring to take the JSON from the request body and convert it into a Candidate Java object. You don't send an id because the database generates it automatically (remember `@GeneratedValue` on the model).

Setting	Value
Method	POST
URL	http://localhost:8080/api/candidates
Body type	JSON

Enter the following body. This works, if you need to figure out what it needs to be exactly, have a look at the model. Can you see how they're related? Doing a GET and seeing the JSON that comes back is helpful as well.

```
{
  "name": "Megan Clark",
  "email": "megan.clark@email.com",
  "fieldOfStudy": "Computer Science"
}
```

- Click **Send**
- The response should show your new candidate, but now with an id that was generated by the database
- Do a GET all again to confirm the new candidate appears in the list

EXERCISE 4 – PUT (update) an existing candidate

This calls the `updateCandidate()` method, annotated with `@PutMapping("/{id}")`. It combines both things you've seen: the `@PathVariable` to know which candidate to update, and the `@RequestBody` to know what to change. You need to send the complete object, not just the fields you want to change.

Setting	Value
Method	PUT
URL	http://localhost:8080/api/candidates/1
Body type	JSON

- Enter the correct body (we're changing the field of study).
- Click **Send**
- The response should show the updated candidate
- Do a GET by ID to verify the change was saved

EXERCISE 5 – DELETE a candidate

This calls the `deleteCandidate()` method, annotated with `@DeleteMapping("/{id}")`. No body is needed — the ID in the URL is enough to know which candidate to remove. Notice that the return type is `ResponseEntity<Void>` — there's no data to return, just a confirmation that the delete was successful.

Setting	Value
Method	DELETE
URL	http://localhost:8080/api/candidates/1
Body	none

- Click **Send**
- You should get a **204 No Content** response — this means the delete was successful but there's nothing to return
- Do a GET all to confirm the candidate is gone
- Try deleting the same ID again. What happens?

OVERVIEW

Look back at the controller code and notice the pattern:

What you did	HTTP Method	URL	Controller annotation	Has body?
Get all	GET	/api/candidates	@GetMapping	No
Get one	GET	/api/candidates/{id}	@GetMapping("/{id}")	No
Create	POST	/api/candidates	@PostMapping	Yes
Update	PUT	/api/candidates/{id}	@PutMapping("/{id}")	Yes
Delete	DELETE	/api/candidates/{id}	@DeleteMapping("/{id}")	No

This is a standard CRUD REST API pattern. You'll see it in almost every Spring Boot REST application.

BONUS CHALLENGE

Now do the same for the **`/api/internships`** endpoints. Look at the `InternshipController` and the `Internship` model to figure out the correct JSON body for POST and PUT requests. Try all five operations.

Module 2

Changing the Spring Boot App

Section 2–1

Changing the Controller

Modifying an Endpoint

- **When requirements change, or while implementing features, we need to change endpoints, for example**
  - Add new features
  - Change URL structure
  - Add query parameters
  - Change response format

- **We can for example, alter the get all internships endpoint to possibly filter on location**

```
@GetMapping
public ResponseEntity<List<Internship>> getAllInternships(
        @RequestParam(required = false) String location) {

    List<Internship> internships;

    if (location != null) {
        internships =
          internshipService.getInternshipsByLocation(location);
    } else {
        internships = internshipService.getAllInternships();
    }

    return ResponseEntity.ok(internships);
}
```

- **This won't compile just yet, because of the service method… But this endpoint will allow the following two calls:**
  - `GET /api/internships` → all internships
  - `GET /api/internships?location=Amsterdam` → only Amsterdam internships

- **Breakdown:**
  - `@RequestParam`: Extract value from query string (behind the ? in the URL)
  - `required = false`: Parameter is optional
  - `if (location != null)`: Check if parameter was provided
  - Call different service methods based on parameter

- **Sometimes it's enough to change the controller, for example when you are changing the status code or when you are changing how you call an existing method in the service**

- **In our case, the service method doesn't exist yet…**

Making Changes to a Service Implementation

- **We need to change the service when implementing features, especially when new business rules are added or when they change:**
  - Filtering data differently
  - Adding calculations
  - Changing behavior based on configuration
  - Different exception/error handling

- **Let's look at this code, what do we need to change if we want to also return unpublished internships?**

```
public List<Internship> getAllInternships() {
        return internshipRepository.findAll().stream()
                .filter(i -> i.isPublished())
                .collect(Collectors.toList());
}
```

- **To make our previous example work, we need to add the location filter, for example like this:**

```
public List<Internship> getInternshipsByLocation(String
location) {
        return internshipRepository.findAll().stream()
                .filter(i -> i.getLocation().equals(location))
                .collect(Collectors.toList());
}
```

Exercise – Changing an Endpoint

In this exercise, you'll modify an existing endpoint to support optional filtering. Right now, `GET /api/candidates` always returns *all* candidates. You're going to change it so that users can optionally filter by field of study.

When you're done, the endpoint should support both:

- `GET /api/candidates` → returns all candidates (same as before)
- `GET /api/candidates?fieldOfStudy=Computer Science` → returns only candidates studying Computer Science

STEP 1 - Modify the controller

Open `CandidateController` and find the `getAllCandidates()` method. Right now it looks like this:

```
@GetMapping
public ResponseEntity<List<Candidate>> getAllCandidates() {
    List<Candidate> candidates = candidateService.getAllCandidates();
    return ResponseEntity.ok(candidates);
}
```

Change it to accept an optional query parameter:

```
@GetMapping
public ResponseEntity<List<Candidate>> getAllCandidates(
        @RequestParam(required = false) String fieldOfStudy) {

    List<Candidate> candidates;

    if (fieldOfStudy != null) {
        candidates = candidateService.getCandidatesByFieldOfStudy(fieldOfStudy);
    } else {
        candidates = candidateService.getAllCandidates();
    }

    return ResponseEntity.ok(candidates);
}
```

Let's break down what changed:

- `@RequestParam(required = false) String fieldOfStudy` tells Spring to look for a query parameter called `fieldOfStudy` in the URL, but not to throw an error if it's missing
- If someone calls `/api/candidates?fieldOfStudy=Biology`, the variable `fieldOfStudy` will be `"Biology"`
- If someone calls `/api/candidates` without the parameter, `fieldOfStudy` will be `null`
- We use a simple `if/else` to decide which service method to call

This won't compile yet, because the method `getCandidatesByFieldOfStudy` doesn't exist in the service. That's your next step.

STEP 2 - Add the method to the service

Open `CandidateService`. You need to add a new method that filters candidates by their field of study.
Add this method to the service class:

```
public List<Candidate> getCandidatesByFieldOfStudy(String fieldOfStudy) {
    return candidateRepository.findAll().stream()
            .filter(c -> c.getFieldOfStudy().equalsIgnoreCase(fieldOfStudy))
            .collect(Collectors.toList());
}
```

A few things to note:

- We're reusing `candidateRepository.findAll()` to get all candidates from the database
- `.stream()` lets us use Java's stream API to filter the list
- `.equalsIgnoreCase()` makes the comparison case-insensitive, so "math" and "Math" both match
- `.collect(Collectors.toList())` turns the filtered stream back into a list

If `Collectors` shows as red in IntelliJ, add this import at the top of the file:

```
import java.util.stream.Collectors;
```

STEP 3 - Test it

Make sure your application is running (restart it if needed, since you changed the code).

Open Insomnia and try these requests:

1. `GET http://localhost:8080/api/candidates`
   - Should return all candidates, same as before
2. `GET http://localhost:8080/api/candidates?fieldOfStudy=Computer Science`
   - Should return only candidates studying Computer Science
   - If no candidates have that field of study, you'll get an empty list `[]`
3. Try filtering on other fields of study that exist in your data
4. Also test it in your browser — you can paste the full URL including the `?fieldOfStudy=...` part directly in the address bar

REFLECTION

- Why did we need to change both the controller and the service?
- What would happen if a candidate has null as their field of study? Would the filtering still work, or would it crash? (Hint: think about what happens when you call .equalsIgnoreCase() on null)
- How would you add a second optional filter, for example filtering by name? What would the URL look like with two filters?

Commit and push your code!

Changing the Data Model

- **Data requirements can change; it can be necessary for a new feature to**
  - Add new fields
  - Change field types
  - Add validation rules
  - Add relationships
- **Let's say we want to track when internships were created**
- **We open up the model and add the new field, plus the getters and setters:**

```
private LocalDateTime createdAt;

public LocalDateTime getCreatedAt() {
    return createdAt;
}

public void setCreatedAt(LocalDateTime createdAt) {
    this.createdAt = createdAt;
}
```

- **This will not automatically add the date, in order to do that we'll have to change the `createInternship` method in the service, we can add this line before saving:**

```
internship.setCreatedAt(LocalDateTime.now());
```

Change a Config Value

- **The Spring Boot app is configured in a specific way, if you open the file `application.properties`, you'll for example see this one:**

```
internships.auto-publish=false
```

- **If you change `false` to `true`, the value of `autoPublish` will change, because that's where the config value is used:**

```
@Value("${internships.auto-publish}")
private boolean autoPublish;
```

- **What happens when you change a config value, depends on what the config value is for**

- **There's a lot of framework configurations in here that can have specific values**
  - You don't need to worry about those for now, we'll introduce you to them whenever you need to know about them

Module 3

Extending the App

Section 3–1

Adding Functionality

Adding an Endpoint

- **When we need to create new features for the API, we often need a new URL. For example when we want to add:**
  - Search functionality
  - Filtering
  - Custom actions
  - Calculations
  - Reports
- **In order to add an endpoint, we need to add a method with the special annotation to the controller**
- **Let's add the endpoint for searching by company:**

```
// iS should be internshipService

@GetMapping("/search/company/{company}")
public List<Internship> searchByCompany
(
   @PathVariable String company
 ) {
    List<Internship> results = iS.searchByCompany(company);
    return results;
}
```

- **When adding a new feature, you often need to change all the layers**
  - In this case we are using a method named `searchByCompany` that we'll have to create in the service

Adding to the service

- **Whenever we have a new piece of functionality, we often need to add a method to the service**
  - **Note**: this is not always true, sometimes we can reuse the existing methods
- **In this case, we don't have a method that's filtering for companies**
- **We can add it to our InternshipService:**

```
public List<Internship> searchByCompany(String company) {
  return internshipRepository.findAll().stream()
     .filter(i -> i.getCompany().contains(company))
     .filter(Internship::isPublished)
     .collect(Collectors.toList());
}
```

- **There are many alternative ways to implement it, we could have called the `getAllInternships` methods first or we could have added a special method to filter by company to the repository instead**

Exercise – Adding Functionality

In this exercise, you'll add a brand new endpoint to the `CandidateController` that allows searching for candidates by name. You'll need to make changes in both the controller and the service, just like the `searchByCompany` example you just saw for internships.

Users want to be able to search for candidates by name. For example, searching for "Megan" should return all candidates whose name contains "Megan". The search should not be case-sensitive, so "megan", "Megan", and "MEGAN" should all give the same results.

STEP 1 – Plan it out

Before writing any code, think about what you need:

- What URL would make sense for this endpoint? (Look at the internship example: `/search/company/{company}`)
- Which HTTP method should it use? You're reading data, not creating or changing anything.
- Which layer do you need to change first, the controller or the service?
- Will you need to change the repository or the model?

STEP 2 – Add the endpoint to the controller

Open `CandidateController` and add a new method. Use the internship example as a guide, but adapt it for candidates and names:

- The URL should be `/search/name/{name}`
- Use `@GetMapping` with that path
- Use `@PathVariable` to capture the name from the URL
- Call a method on the `candidateService` that doesn't exist yet — you'll create it in the next step

If you're stuck, here's the structure to start with:

```
@GetMapping("/search/name/{name}")
public ResponseEntity<List<Candidate>> searchByName(@PathVariable String name) {
    // call the service and return the result
}
```

STEP 3 – Add the method to the service

Open `CandidateService` and add a new method called `searchByName`. Look at the internship example for the pattern:

```
// Internship example for reference:
public List<Internship> searchByCompany(String company) {
    return internshipRepository.findAll().stream()
        .filter(i -> i.getCompany().contains(company))
        .filter(Internship::isPublished)
        .collect(Collectors.toList());
}
```

Your method should:

- Get all candidates from the repository
- Use `.stream()` and `.filter()` to keep only candidates whose name contains the search term
- Make it case-insensitive (hint: what if you convert both the candidate's name and the search term to lowercase before comparing?)
- Collect the results back into a list

Give it a try on your own first. If you get stuck, here's a hint for the case-insensitive comparison:

```
.filter(c -> c.getName().toLowerCase().contains(name.toLowerCase()))
```

STEP 4 – Test it

Restart the application and open Insomnia.

1. First, make sure you have some candidates in the database — do a `GET http://localhost:8080/api/candidates` to check
2. Now try your new endpoint: `GET http://localhost:8080/api/candidates/search/name/megan`
3. Try searching for just part of a name, for example the first few letters
4. Try different casings — does `Megan`, `megan`, and `MEGAN` all give the same result?
5. Try a name that doesn't exist — you should get an empty list `[]`

STEP 5 – Add another endpoint on your own

Now add a second search endpoint without a guided walkthrough. Add an endpoint that searches candidates by email. Think about:

- What URL will you use?
- Should it be an exact match or a partial match (contains)?
- What changes do you need in the controller?
- What changes do you need in the service?

Test it in Insomnia when you're done.

Think about it

- You now have three GET endpoints in your controller: get all, get by ID, and search by name (and maybe search by email). How does Spring know which method to call when a request comes in? (Hint: look at the URL patterns, are any of them ambiguous?)
- In the search by name endpoint, what would happen if a candidate's name is `null` in the database? How could you prevent a crash?
- Can you think of a situation where you'd want to add an endpoint to the controller but *wouldn't* need to add a new method to the service?

Commit and push your code!

Modifying the data model

- **For this feature, we don’t need to change the data model, because the company field is already existing**

- **But changing the data model is not always straightforward, for example:**
  - What do you do with older records that don’t have that column?
  - What do you do with new validation rules and old data?

- **This would require database migrations**
  - that isn't currently in scope for now

- **What we do, is work with this configuration in `application.properties`. This automatically updates the database when we change the model:**

```
spring.jpa.hibernate.ddl-auto=update
```

Exercise – Adding to the Data Model

In these exercises, you'll add a new field to the `Candidate` model.

A new requirement has come in: we need to track **when each candidate was registered** in the system. Right now the `Candidate` model only has `id`, `name`, `email`, and `fieldOfStudy` — there's no timestamp.

STEP 1 - Change the model

Open the `Candidate` class in the `model` package and add a new field:

```
private LocalDateTime registeredAt;
```

If `LocalDateTime` shows as red, add this import at the top:

```
import java.time.LocalDateTime;
```

Now add a getter and a setter for the new field:

```
public LocalDateTime getRegisteredAt() {
    return registeredAt;
}

public void setRegisteredAt(LocalDateTime registeredAt) {
    this.registeredAt = registeredAt;
}
```

That's it for the model. Because we have `spring.jpa.hibernate.ddl-auto=update` in our `application.properties`, Spring will automatically add a new column to the `candidates` table in the database when we restart the application.

STEP 2 - Set the timestamp in the service

Adding the field to the model doesn't mean it gets filled in automatically. If you create a new candidate right now, `registeredAt` would be `null`. We need to set it ourselves.

Open `CandidateService` and find the `createCandidate` method. Add one line before the save:

```
public Candidate createCandidate(Candidate candidate) {
    candidate.setRegisteredAt(LocalDateTime.now());
    return candidateRepository.save(candidate);
}
```

Make sure to add the import for `LocalDateTime` in the service as well if it's not there yet.

STEP 3 - Test it

Restart the application and use Insomnia to:

1. Create a new candidate with a POST request (same body as before — you don't include `registeredAt` in the JSON)
2. Look at the response — you should see `registeredAt` with the current date and time
3. Do a GET all — candidates that existed before your change will have `registeredAt: null`, but new ones will have a timestamp

Exercise – Changing the Model and Working with Config Values

Right now, every candidate is immediately visible in the system when created.

A new business rule says: **candidates should be hidden by default** until they're reviewed, but we want to be able to easily turn this behavior on or off without changing code.

This is a perfect use case for a configuration value.

STEP 1 - Add a field to the model

Open the `Candidate` model and add:

```
private boolean visible;
```

And the getter and setter:

```
public boolean isVisible() {
    return visible;
}

public void setVisible(boolean visible) {
    this.visible = visible;

}
```

STEP 2 - Add the config value

Open `application.properties` and add this line:

```
candidates.visible-by-default=false
```

STEP 3 - Use the config value in the service

Open `CandidateService` and add this field at the top of the class, above the constructor:

```
@Value("${candidates.visible-by-default}")
private boolean visibleByDefault;
```

If `@Value` shows as red, add this import:

```
import org.springframework.beans.factory.annotation.Value;
```

Now update the `createCandidate` method to use this config value:

```
public Candidate createCandidate(Candidate candidate) {
    candidate.setRegisteredAt(LocalDateTime.now());
    candidate.setVisible(visibleByDefault);
    return candidateRepository.save(candidate);
}
```

STEP 4 - Test it

Restart the application and use Insomnia to:

1. Create a new candidate with a POST request
2. Check the response — `visible` should be `false` (because that's what the config says)
3. Now open `application.properties` and change the value to `true`:

```
candidates.visible-by-default=true
```

4. Restart the application and create another candidate
5. This time, `visible` should be `true`

Notice that you changed the behavior **without modifying any Java code** — only the config file. This is very powerful for settings that might differ between environments (development, testing, production) or that you want to toggle without a code change.

Think about it

- What would you change if you wanted the `getAllCandidates` endpoint to only return visible candidates? Which layers would you need to modify?
- What happens to candidates that were created before you added the `visible` field? What is their value? (Hint: think about default values for boolean in Java)
- Can you think of other settings in the internship app that would make sense as configuration values rather than hardcoded in the code?

Commit and push your code!

Understanding the request flow in MVC

- **Let's get back to the request flow**
- **MVC or Model View Controller is a common pattern to control the flow of data in an API**
- **In our case, The V in MVC is the JSON and the C is split up into a controller and a service**
- **Let's assume the following will happen, a user uses the frontend to create an internship**
  - The user clicks the "Create Internship" button on the frontend
  - The frontend collects form data and sends the following request:

```
POST http://localhost:8080/api/internships
Content-Type: application/json

{
  "title": "Java Developer Intern",
  "company": "Tech Corp",
  "description": "Build Spring Boot applications",
  "location": "Amsterdam",
  "published": true
}
```

    * Spring's `DispatcherServlet` receives HTTP request
    * It looks at URL: `/api/internships`
    * It at method: `POST`
    * It finds and triggers the matching controller method (`createInternship`)
    * This controller calls the services method (also named `createInternship`)

Understanding the request flow in MVC

- The service calls the repository layer, and then JPA:
  - Converts Java object to SQL
  - Generates: `INSERT INTO internships (title, company, ...) VALUES (?, ?, ...)`
  - Executes SQL against MySQL database
- The database then:
  - Stores the data
  - Auto-generates ID
  - Returns saved record with ID
- And then the response flows back from the database, to JPA, to the repository, to the service, then to the controller and there Spring converts Java object to JSON and we get a response like this in our example:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 25,
  "title": "Java Developer Intern",
  "company": "Tech Corp",
  "description": "Build Spring Boot applications",
  "location": "Amsterdam",
  "published": true
}
```

- The frontend receives the response and shows a success message, refreshes the internship list and displays the new card with the new data

Module 4

Testing the Layers of a Spring Boot App

Section 4–1

Testing the Controllers

Creating Controller Tests

- **We have to write tests for our controllers to**
  - Verify endpoints work correctly
  - Check HTTP status codes
  - Validate JSON responses
  - Test error handling
  - Catch breaking changes early
- **A few things to keep in mind when testing controllers**
  - Don't hit the real database
  - Do use mock services
    - These are fake services, "mocking" what would be a real response from a real database
    - This is made easier with dependency injection
  - Focus on HTTP layer behavior
  - Test request → response mapping
- **Spring things to use to create our tests**
  - `@WebMvcTest`: Loads only web layer (not full application)
  - `MockMvc`: Simulates HTTP requests
  - `@MockitoBean`: Creates fake service (we control behavior and are testing the controller layer only)

Exercise – Adding Controller Tests

In this exercise, you'll write controller tests for the `CandidateController`. You won't be starting from scratch because there's already a complete test class for `InternshipController` that you'll use as your guide.

STEP 1 – Study the existing tests

Open `InternshipControllerTest` in the test folder and read through it carefully. Before writing any code, answer these questions for yourself:

1. What does `@WebMvcTest(InternshipController.class)` do? Why don't we load the entire application?
2. What is `MockMvc` used for? Is it making real HTTP requests?
3. Why is `InternshipService` annotated with `@MockitoBean` instead of `@Autowired`? What would happen if we used the real service?
4. In the `getAllInternships` test, what does `when(...).thenReturn(...)` do? Why do we need it?
5. Look at the `jsonPath` checks — how do they map to the JSON response? What does `$[0].title` mean? What does `$.length()` mean?
6. In the `deleteInternship` test, what does `verify(internshipService, times(1)).deleteInternship(id)` check?

Take your time with this because understanding these tests is more important than rushing to write your own.

STEP 2 – Plan your test cases

Before you write code, think about what you need to test. The `CandidateController` has these endpoints:

* `GET /api/candidates` → get all
* `GET /api/candidates/{id}` → get one by ID
* `POST /api/candidates` → create a candidate
* `PUT /api/candidates/{id}` → update a candidate
* `DELETE /api/candidates/{id}` → delete a candidate

Write down on paper which test cases you think are needed. Here are some hints to get you thinking:

- What should happen when you get all candidates and there are candidates in the database?
- What should happen when you get all candidates but there are none?
- When you create a candidate, what should the response contain?
- When you delete a candidate, what status code do you expect?
- When you update a candidate, how do you verify the updated data comes back?

Aim for **at least** 4 test cases before moving on. (You don't need to code them yet.)

STEP 3 – Create the test class

Create a new test class called `CandidateControllerTest` in the test folder, under the `controller` package (next to `InternshipControllerTest`).

Start with the class setup. Use the internship test as your reference, but swap out the internship-specific parts for candidate ones:

```
@WebMvcTest(CandidateController.class)
class CandidateControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private CandidateService candidateService;

    // your tests go here
}
```

Make sure your imports match what you need. Look at the imports in `InternshipControllerTest` — you'll need most of the same ones, plus the imports for `Candidate`, `CandidateController`, and `CandidateService`.

STEP 4 – Write the tests

Now implement your test cases. Here are some pointers to keep in mind:

For testing GET all candidates:

- Create a couple of Candidate objects as test data (remember: name, email, fieldOfStudy)
- Use `when(candidateService.getAllCandidates()).thenReturn(...)` to control what the mock returns
- Use jsonPath to check that the right data comes back
- What jsonPath would you use to check the first candidate's email?

For testing POST (create):

- You need to send a JSON body with the request, look at how the internship test does this with `.content("""...""")`
- What fields does a candidate need in the JSON body?
- The response should include an `id` that the database would have generated, set this on your `savedCandidate` test object

For testing DELETE:

- This one is the most similar to the internship example
- What status code does the `deleteCandidate` method return? (Look back at the controller code)

For testing GET by ID:

- This one isn't in the internship tests, you'll need to figure it out yourself
- Hint: the URL pattern is `/api/candidates/{id}`, look at how the delete test passes the ID into the URL
- What mock setup do you need? Which service method is called?

STEP 5 – Run the tests

Right-click on CandidateControllerTest and select Run. If tests fail, read the error messages carefully:

- `404` usually means the URL in your test doesn't match the controller
- `400` often means the JSON body is missing or malformed

  Assertion errors tell you the response didn't match your expectation. If that happens, compare what you expected with what came back

Think about it

- The controller tests don't touch the database at all. Why is that a good thing?
- If someone changes the URL of an endpoint in the controller, which would catch it first: the controller test or the service test?
- We didn't write a test for "what happens when you request a candidate with an ID that doesn't exist." Why might that be tricky with the current controller code? (Hint: look at what `getCandidateById` does when the candidate isn't found)

Commit and push your code!

Section 4–2

Testing the Service Layer

Writing Tests for the Service Layer

- **We need to test the service layer to**
  - Verify business logic
  - Test decision-making
  - Check edge cases
  - Ensure rules are enforced

- **A few things to keep in mind when testing the service layer**
  - Don't use real database
  - Do mock repositories
  - Focus on business logic
  - Test all branches (if/else)

- **Annotations that you'll see when testing the service layer**
  - `@ExtendWith(MockitoExtension.class)` to enable Mockito
  - `@Mock` to create mock objects
  - `@InjectMocks` to inject mocks into service

Exercise – Adding Service Tests

In this exercise, you'll write tests for the `CandidateService`. The service layer is where the business logic lives, so these tests focus on verifying that logic. This means our focus is not on HTTP requests, not on database queries, just the decisions and rules in between.

And you have a completed example to study first: `InternshipServiceTest`.

STEP 1 – Study the existing tests

Open `InternshipServiceTest` and read through it carefully. Answer these questions for yourself:

1. How is the setup different from the controller test? You'll notice `@ExtendWith(MockitoExtension.class)` instead of `@WebMvcTest`, why do you think that is?
2. What's the difference between `@Mock` and `@MockitoBean`? (Hint: one is for Spring, one is for plain Mockito)
3. What does `@InjectMocks` do? Which object gets the mocked dependencies injected?
4. Look at the `getAllInternships_shouldReturnOnlyPublished` test. It creates 3 internships but expects only 2 back. What business logic is being tested here?
5. Look at the two `createInternship` tests. They test the same method but with different config values. What does `ReflectionTestUtils.setField(...)` do and why is it needed? (Hint: think about how `@Value` fields are set in the real application vs. in a test)
6. Notice that there's no `MockMvc` and no HTTP requests anywhere, we're calling the service methods directly. Why does that make sense?

STEP 2 – Review the CandidateService

Before planning your tests, look at the `CandidateService` code and think about what business logic it contains. Consider the methods and features you've built up during the previous exercises:

- `getAllCandidates()` → returns all candidates
- `getCandidateById(Long id)` → returns one candidate, throws exception if not found
- `createCandidate(Candidate candidate)` → sets `registeredAt` and `visible` (based on config), then saves
- `updateCandidate(Long id, Candidate updated)` → finds existing candidate, updates fields, saves
- `deleteCandidate(Long id)` → deletes by ID
- `getCandidatesByFieldOfStudy(String fieldOfStudy)` →filters candidates by field of study (case-insensitive)
- `searchByName(String name)` → filters candidates by name (case-insensitive)

STEP 3 – Plan your test cases

Write down test cases on paper before coding. Here are some hints:

For `getAllCandidates`:

- What if there are 3 candidates? Does the service return all 3?
- What if there are no candidates at all?

For `createCandidate`:

- Does `registeredAt` get set? (It should not be `null` after creation)
- When `visibleByDefault` is `false`, is the candidate created with `visible = false`?
- When `visibleByDefault` is `true`, is the candidate created with `visible = true`?

- Look at how the internship tests use `ReflectionTestUtils.setField` for the config value — you'll need the same approach for `visibleByDefault`

For `getCandidateById`:

- What happens when the candidate exists?
- What happens when the candidate does *not* exist? (Hint: the method throws a `RuntimeException` — how do you test that an exception is thrown?)

For `getCandidatesByFieldOfStudy`:

- If there are 3 candidates but only 2 study "Computer Science", does the method return exactly those 2?
- Does the case-insensitive matching work? If you search for "computer science" (lowercase), does it still find "Computer Science"?

Aim for **at least** 5 test cases.

STEP 4 – Create the test class

Create `CandidateServiceTest` in the test folder under the `service` package. Start with the setup:

```
@ExtendWith(MockitoExtension.class)
class CandidateServiceTest {

    @Mock
    private CandidateRepository candidateRepository;

    @InjectMocks
    private CandidateService candidateService;

    // your tests go here
}
```

STEP 5 – Write the tests

Here are some pointers for the trickier test cases:

Testing that `registeredAt` is set on creation: You can't check the exact timestamp, but you can verify it's not `null`. However, there's a subtlety. The `when(...).thenReturn(...)` returns whatever you tell it to, not the actual object that was passed to `save()`. You may need to use `any()` and think about what you're really asserting. One approach is to use `verify` to check that `save` was called, and use an ArgumentCaptor to inspect what was passed:

```
// This is a more advanced technique,  feel free to use a simpler approach first
verify(candidateRepository).save(argThat(candidate ->
    candidate.getRegisteredAt() != null
));
```

Testing that an exception is thrown: JUnit 5 has a clean way to test exceptions:

```
@Test
void getCandidateById_whenNotFound_shouldThrowException() {
    when(candidateRepository.findById(99L)).thenReturn(Optional.empty());

    assertThrows(RuntimeException.class, () -> {
        candidateService.getCandidateById(99L);
    });
}
```

Make sure to add `import java.util.Optional;` if needed.

Testing the config value (visible by default): Follow the internship pattern and write two tests for the same method with different config values:

```
ReflectionTestUtils.setField(candidateService, "visibleByDefault", true);
```

STEP 6 – Run the tests

Run `CandidateServiceTest` and fix any failures. Common issues:

- Forgetting to set up the mock with `when(...).thenReturn(...)`, your service will get null back from the repository
- Forgetting to use `ReflectionTestUtils.setField` for the config value, it will default to `false`
- Import issues? Make sure `Optional`, `ReflectionTestUtils`, and all Mockito static imports are present

Think about it

- The controller tests mock the service, and the service tests mock the repository. What does this tell you about how we test in layers?
- In the `createCandidate` tests, we're testing that the service sets `registeredAt` and `visible` correctly. Why is this a service test and not a controller test?
- If you changed the filtering logic in `getCandidatesByFieldOfStudy` from case-insensitive to case-sensitive, which test should catch that regression?

Commit and push your code!

Section 4–3

Testing the Data Layer

Testing the Data Layer

- **We need to test the data layer to:**
  - Test relationships
  - Ensure data integrity
  - Verify custom queries work
- **A few things to keep in mind:**
  - Do use real database (H2 in-memory for tests)
  - Verify JPA mappings
  - Check cascading operations
- **We'll use `@DataJpaTest` to load only JPA components**

Exercise – Adding Data Layer Tests

In this exercise, you'll write tests for the `CandidateRepository`.

These tests are different from what you've done so far. They use a **real database**.

Instead of mocking, Spring spins up an in-memory H2 database just for testing, so you can verify that your entities are correctly mapped, data is actually saved, and queries return what you expect.

STEP 1 – Study the existing tests

Open `InternshipRepositoryTest` and read through it. Answer these questions for yourself:

1. What does `@DataJpaTest` do? How is it different from `@WebMvcTest` and `@ExtendWith(MockitoExtension.class)`? (Hint: look at what gets loaded. Is it the full application, just the web layer, or just the data layer?)
2. Notice there are **no mocks** in this test. The `InternshipRepository` is `@Autowired`, not mocked. Why does that make sense for data layer tests?
3. What is `TestEntityManager`? Why do some tests use it to insert data instead of using the repository itself? (Hint: if you're testing that `findById` works, you don't want to rely on `save` also working correctly)
4. What does `persistAndFlush` do? Why the "flush" part?
5. Look at the `save_shouldPersistInternship` test. It saves through the repository, then verifies through the `entityManager`. Why use two different ways to write and read?
6. Look at the `delete_shouldRemoveInternship` test. After deleting, it tries to find the entity and asserts it's gone. Why is `Optional` useful here?

STEP 2 – Plan your test cases

Think about what you need to verify about the `Candidate` entity and repository. The candidate has these fields: `id`, `name`, `email`, `fieldOfStudy`, `registeredAt`, and `visible`. **Aim to create atleast 4 test cases.**

Basic CRUD operations:

- Can you save a candidate and get a generated ID back?
- Can you find a candidate by ID after persisting it?
- If you persist 3 candidates, does `findAll` return all 3?
- If you delete a candidate, is it actually gone?

Field mapping:

- When you save a candidate with all fields filled in, are all fields correctly stored and retrieved? (This will verify your `@Entity` mapping is correct for every field, including the ones you added like `registeredAt` and `visible`)

Edge cases:

- What happens when you search for an ID that doesn't exist?
- Can you update a candidate's fields and save again? Does the update persist?

STEP 3 – Create the test class

Create `CandidateRepositoryTest` in the test folder under the `repository` package. Start with the setup:

```
@DataJpaTest
class CandidateRepositoryTest {

    @Autowired
    private CandidateRepository candidateRepository;

    @Autowired
    private TestEntityManager entityManager;

    // your tests go here
}
```

Make sure your imports include the same testing imports you see in `InternshipRepositoryTest`.

STEP 4 – Write the tests

Use the internship tests as your template, but remember that candidates have different fields and constructors. Here are some pointers:

For the save test: Follow the same pattern: save through the repository, verify through the entity manager. Check that the ID was generated and that the fields you set are correctly persisted:

```
Candidate candidate = new Candidate("Megan Clark", "megan@email.com", "Computer
Science");
```

Make sure you verify more than just the name, check email and field of study too.

For testing all fields including the new ones: When creating a test candidate, also set the fields you added in the earlier exercises:

```
candidate.setRegisteredAt(LocalDateTime.now());
candidate.setVisible(true);
```

After saving and retrieving, verify these fields are stored correctly. This is valuable because it confirms your JPA mapping works for every column in the table.

For the findById with a non-existent ID: This is a short but important test:

```
Optional<Candidate> found = candidateRepository.findById(999L);
assertFalse(found.isPresent());
```

For testing update: This pattern isn't in the internship example, so you'll need to figure it out. The general approach is:

- Persist a candidate using the entity manager
- Retrieve it through the repository
- Change a field

- Save it again through the repository
- Retrieve it once more through the entity manager
- Assert the field was updated

STEP 5 – Run the tests

Run `CandidateRepositoryTest`. These tests may take slightly longer to start than the other tests because Spring needs to set up the in-memory database.

Common issues:

- Constructor mismatches. Make sure you're creating `Candidate` objects with the right arguments for your constructor
- `registeredAt` being `null`? It happens, remember, in these tests you're not going through the service, so you need to set it yourself if you want to test that the field is persisted
- Missing `@Entity` or `@Id` annotations on the model. Oops, these tests will fail immediately if the JPA mapping is broken

Think about it

- You've now tested all three layers: controller, service, and repository. Each layer mocks the layer below it, except the repository tests which use a real (in-memory) database. Why do you think the data layer is the one where we stop mocking?
- If you renamed a field in the `Candidate` model but forgot to update the getter, which tests would catch that: controller, service, or data layer?
- The in-memory H2 database is empty at the start of every test. Why is that important? What could go wrong if tests shared the same data?

Commit and push your code!